

ESP32 Oregon Transmitter Sensor

Technical Architecture & Hardware Validation Manual

VERSION 0.1.0-beta.1 - BETA / TESTING

Repository: [pgpaolo/esp32-Oregon-Trasmitter-Sensor](#)

Purpose: emulate multiple Oregon Scientific 433.92 MHz sensors from modern sensors connected to an ESP32, with calibration and configuration performed before transmission.

Important: this document describes the initial beta implementation. Compatibility with original WMR88/WMR200 consoles must be confirmed by the staged bench and field tests before the project is considered production-ready.

1. Design objective

The transmitter is intentionally an **Oregon Scientific sensor emulator**, not a custom RF protocol. Physical measurements are acquired by the ESP32, calibrated, mapped to virtual Oregon devices, encoded as Oregon V3 frames and transmitted through a low-cost OOK/ASK 433 MHz module.

The companion LILYGO/SX1278 gateway receives exactly the same Oregon frame that an original WMR88 or WMR200 console is expected to receive. There is no privileged or private frame format for the gateway.

Architecture

```
BME280 / UV / rain / wind inputs -> calibration -> virtual Oregon sensor -> OSV3 encoder -> GPIO OOK waveform -> 433.92 MHz TX -> WMR88/WMR200 and LILYGO gateway
```

2. Hardware

Component	Beta choice	Role
MCU	ESP32 DevKit / ESP32-C3	sensor acquisition, Web UI, NVS, scheduler, encoder
RF TX	FS1000A or equivalent ASK/OOK 433 MHz	simple carrier on/off transmitter
T/RH/P	BME280	temperature, humidity, local pressure
UV	generic analog input initially	UV source with Web calibration
RX test	existing LILYGO ESP32 + SX1278	Oregon frame validation

Recommended ESP32 DevKit pin mapping

Function	Pin	Notes
RF DATA	GPIO25	compile-time configurable
BME280 SDA	GPIO21	I2C
BME280 SCL	GPIO22	I2C
UV ADC	GPIO34	choose a valid ADC pin on C3
Status LED	GPIO2	optional

All grounds must be common. The RF module supply must follow the actual module specification. Never apply 5 V to an ESP32 GPIO. A straight wire around 17.3 cm is a useful quarter-wave starting antenna for 433 MHz bench work.

3. Firmware architecture

The firmware is structured around a virtual-sensor layer. Each virtual sensor has an Oregon model identity, channel, rolling ID, calibration settings, transmission cadence and repeat policy. Configuration persists in ESP32 Preferences/NVS.

Layer	Responsibility
Input	read BME280 / UV and future physical sensors
Calibration	offsets, multipliers, range clamps
Virtual sensor	model + channel + rolling ID + status
Encoder	construct Oregon payload and checksum
RF	Manchester/OOK waveform
Scheduler	serialize several virtual sensors
Web console	setup, preview, manual TX, firmware update

4. Oregon profiles

Model	Sensor ID	Beta status	Primary data
THGR810 / THGN801	F824	first validation target	temperature + humidity
UVN800	D874	second validation target	UV index
PCR800	2914	bench encoder	rain rate + total
WGR800	1984	bench encoder	average/gust wind + direction

Identity fields

Sensor code is fixed by the Oregon model. **Channel** is configured separately. **Rolling ID** is an 8-bit virtual-sensor identity and is persisted across resets so the original console does not need to be paired again after every ESP32 reboot.

5. OSV3 RF layer

Initial beta timing uses a nominal 488 us Manchester half-bit. The Web UI permits 430-550 us so timing can be tuned after comparing the ESP waveform with genuine Oregon captures.

Initial transmission sequence

```
24 logical preamble 1 bits -> sync 0101 -> Oregon payload nibbles, least-significant bit first -> carrier off
```

RF polarity can be inverted because cheap transmitter/baseband combinations may invert the logic. This makes the first logic-analyzer and LILYGO tests much faster.

6. Calibration before transmission

Calibration is applied before Oregon encoding. This is essential: both an original console and the LILYGO gateway must receive the same already-corrected engineering value.

Example

```
BME280 21.63 C -> offset -0.40 C -> 21.23 C -> Oregon normalization -> THGR810 payload 21.2 C
```

Pressure from the BME280 is retained locally. It is not inserted into THGR810 because that would break compatibility with the real Oregon model.

7. Web configuration console

At first boot the firmware starts the access point **Oregon-TX-Setup** and serves the configuration page at **http://192.168.4.1/**. Wi-Fi STA credentials can also be stored.

Configurable item	Purpose
RF enabled / polarity	safe bench mode and baseband inversion
OSV3 half-bit	timing tuning
repeat count / gap	console reception tuning
THGR810 channel / rolling ID	virtual sensor identity
temperature / humidity offsets	pre-encoding calibration
UV channel / rolling ID	UV virtual identity
UV multiplier / offset	sensor calibration
TX intervals	model cadence testing
Transmit now	manual diagnostic burst
Firmware update	browser upload of PlatformIO .bin

8. Validation sequence

Stage 0 - firmware: CI build, Web UI, NVS persistence, OTA and sensor detection.

Stage 1 - baseband: disconnect RF module, capture GPIO DATA on logic analyzer, verify half-bit, preamble, sync, repeat behavior and polarity.

Stage 2 - LILYGO: connect FS1000A; confirm raw frame, checksum, sensor code, channel, rolling ID and engineering values on the existing SX1278 gateway.

Stage 3 - WMR200: enable THGR810 only, pair an unused channel and confirm stable display. Reboot ESP32 and verify identity persistence. Then repeat for UVN800.

Stage 4 - WMR88: repeat the same tests and observe remote-sensor/channel limits.

Stage 5 - multi-sensor: enable THGR810 + UVN800 together and verify scheduler serialization and 24-hour stability.

Stage 6 - PCR800/WGR800: only after comparing generated frames with genuine captures; verify BCD ordering, units, checksum position, cadence and status bits.

9. Beta exit criteria

Requirement	Target
CI	green on esp32dev and esp32c3
THGR810	validated on LILYGO + WMR200 + WMR88
UVN800	validated on LILYGO + at least one original console
Soak test	72 hours mixed-profile operation
Identity	no spontaneous rolling-ID changes
RF timing	measured and documented
Captures	at least one stored raw capture per validated profile

10. Repository and licensing

Source repository: **pgpaolo/esp32-Oregon-Trasmitter-Sensor**. Companion receiver: **pgpaolo/esp32-oregon-technoline-weather-gateway**.

Project license: **GNU LGPLv3-or-later**. Public protocol references and prior-art links are maintained in **docs/REFERENCES.md**.

Document generated for the initial 0.1.0-beta.1 testing baseline - 1 September 2026.